# K-Nearest Neighbours (KNN)

A simple guide to the idea, the distance measures, choosing k, and the pros and cons



# What Is KNN?

> *“If it walks like a duck and quacks like a duck, then it’s probably a duck.”*
> KNN labels something new by looking at the examples most similar to it.



## Supervised

Learns from labelled examples (data whose answer is already known).



## Classification & regression

Classification: the neighbours vote.
Regression: take the average of the neighbours’ values.



## “Lazy” learner

No model is built in training. It just stores the data and does the work when a new point arrives.



## k = how many neighbours

k is the number of nearest neighbours we look at. It is the main setting to choose.

# How KNN Works: 5 Simple Steps



1

## Store

Keep all the labelled training examples.

›

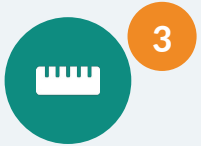


2

## Choose k

Decide how many neighbours to look at (e.g. k = 3).

›

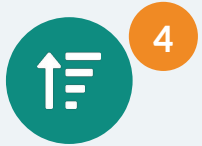


3

## Measure

Find the distance from the new point to every training point.

›



4

## Pick k nearest

Sort by distance and keep the k closest points.

›



5

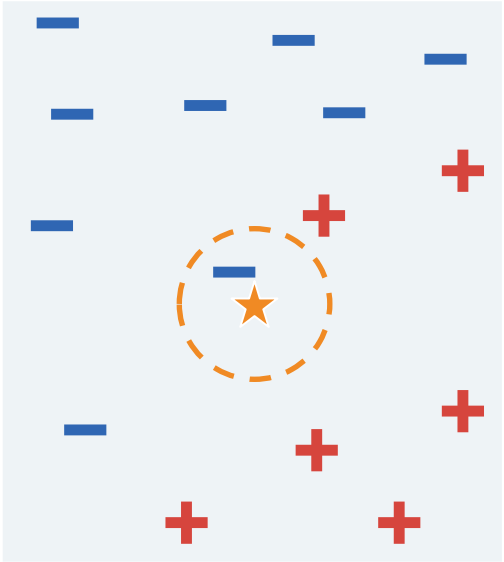
## Vote

The most common label among them is the answer.

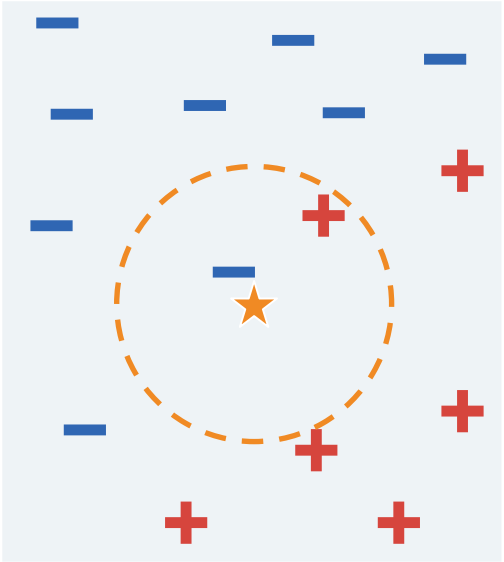
**For regression** (predicting a number, e.g. house price): instead of voting, take the average of the k neighbours' values.

# Nearest Neighbours in a Picture

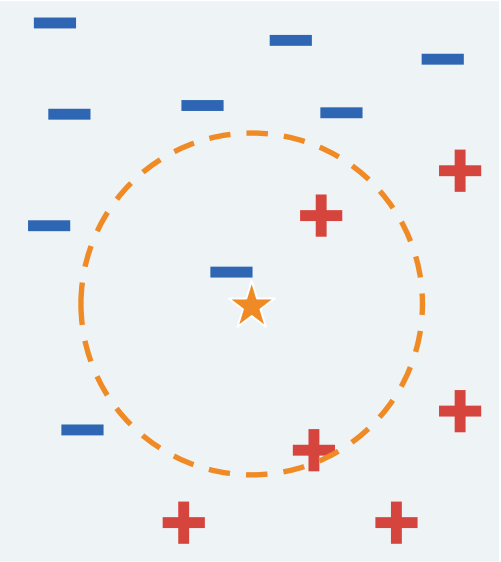
The k nearest neighbours of a new point ★ are the k training points closest to it. The dashed circle shows who gets a vote.



**k = 1:** Nearest point is "–"
→ **Class –**

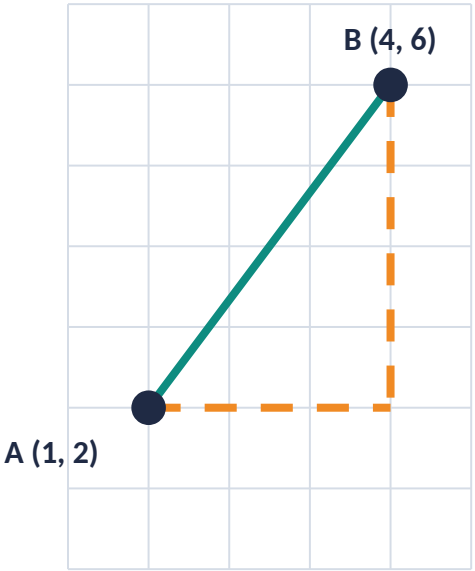


**k = 2:** One "–" and one "+"
→ **Tie! (why we use odd k)**



**k = 3:** Two "+" and one "–"
→ **Class +**

# Distance Measures: How “Close” Is Close?




— Euclidean = 5
- - Manhattan = 3 + 4 = 7

## Euclidean

Straight-line distance (“as the crow flies”). The most common choice.

$\sqrt{((x_1-y_1)^2 + (x_2-y_2)^2)}$

## Manhattan

Walk along the grid, like city blocks. Add up the differences.

$|x_1-y_1| + |x_2-y_2|$

## Minkowski

A general family: p = 1 gives Manhattan, p = 2 gives Euclidean.

*one formula, adjustable p*

## Hamming

For categories (yes/no, colours): count how many features differ.

*number of mismatches*

Others exist too (Chebyshev, Canberra, Mahalanobis…), but Euclidean and Manhattan are enough to start.

# Worked Example: T-Shirt Size

**New customer:** height 161 cm, weight 61 kg → size M or L?

| Rank | Height | Weight | Size | Distance |
|---|---|---|---|---|
| 1 | 163 | 61 | M | 2.00 |
| 2 | 160 | 59 | M | 2.24 |
| 3 | 160 | 64 | L | 3.16 |
| 4 | 163 | 64 | L | 3.61 |
| 5 | 165 | 61 | L | 4.00 |
| 6 | 158 | 58 | M | 4.24 |
| 7 | 157 | 64 | M | 5.00 |
| 8 | 165 | 65 | L | 5.66 |

Sorted by Euclidean distance to (161, 61). Green = 3 nearest, orange = next 2.

**k = 3**

Neighbours: M, M, L → 2 M vs 1 L

**Prediction: M**

**k = 5**

Neighbours: M, M, L, L, L → 2 M vs 3 L

**Prediction: L**

**Lesson:** same data, different k → different answer. Choosing k matters!

# Choosing the Value of k

### k too small (e.g. k = 1)

Listens to just one or two points, so one noisy or wrong point can change the answer. Boundary is jagged (overfitting).

### k just right

Enough neighbours to ignore noise, but still local enough to follow the real pattern.

### k too large (e.g. k = N)

The neighbourhood pulls in points from other classes; the answer drifts to the biggest class (underfitting).

## Practical tips

1. **Use an odd k** for two classes, so votes can't tie.
2. **Start with k ≈ √N,** where N = number of training points (e.g. 100 points → k ≈ 10, use 9 or 11).
3. **Try a few values** and keep the one that does best on test (validation) data.

# Advantages and Disadvantages

## Advantages

- Very simple to understand and implement
- No training phase: just store the data
- Works for classification and regression
- Handles many classes naturally
- Can follow curvy, non-linear boundaries
- New data can be added any time

## Disadvantages

- Slow to predict on big datasets (compares with every point)
- Needs memory to store all the data
- Must pick a good k and distance measure
- Sensitive to noise and irrelevant features
- Needs feature scaling (see tip below)
- Struggles with many features (curse of dimensionality)

**Tip – scale your features:** if salary (₹10,000–₹1,00,000) and age (20–60) are used together, salary dominates every distance and age is ignored. Bring all features to a similar range first.